

Cubos Multidimensionales: pipeline perezoso sobre ERA5-Land en la CGSM

Ejercicios 1 y 2 — Capítulo 20

Lina María Quintero Fonseca

2026-04-25

Tabla de contenidos

1	Introducción	2
2	Ejercicio 1 — Adquisición de datos climáticos (Copernicus CDS)	2
2.1	Configuración de la solicitud	2
2.2	Script de descarga	2
2.3	Verificación del archivo descargado	3
3	Ejercicio 2 — Pipeline perezoso en Python, R y Julia	3
3.1	Lectura y auditoría del cubo	3
3.1.1	Python	3
3.1.2	R	5
3.1.3	Julia	6
3.2	Álgebra de mapas: conversión de Kelvin a Celsius	6
3.2.1	Python	6
3.2.2	R	8
3.2.3	Julia	9
3.3	Inspección de la resolución temporal	9
3.3.1	Python	9
3.3.2	R	9
3.3.3	Julia	10
3.4	Remuestreo temporal: de horario a diario	10
3.4.1	Python	10
3.4.2	R	11
3.4.3	Julia	12
3.5	Máxima mensual y fecha de ocurrencia	12
3.5.1	Python	12
3.5.2	R	14
3.5.3	Julia	15
3.6	Reproyección a MAGNA-SIRGAS / Origen Nacional (EPSG:9377)	15
3.6.1	Python	15
3.6.2	R	17
3.6.3	Julia	18
3.7	Persistencia: exportación a NetCDF	18
3.7.1	Python	18
3.7.2	R	19
3.7.3	Julia	19

4	Resumen sintáctico del pipeline	19
5	Preguntas analíticas	20
5.1	Sobre evaluación perezosa: ¿qué pasa si re proyectamos antes de remuestrear?	20
5.2	Sobre convenciones CF y decodificación temporal	21
6	Conclusiones	21
7	Referencias	21

1 Introducción

El presente documento materializa la solución a los Ejercicios 1 y 2 del capítulo 20 del curso de Programación en SIG, los cuales abordan la adquisición de datos de reanálisis climático y el procesamiento de un hipercubo NetCDF mediante evaluación perezosa. El área de estudio corresponde a la Ciénaga Grande de Santa Marta —CGSM, Magdalena, Colombia—, un complejo lagunar costero cuya dinámica termohídrica reviste un interés analítico de primer orden para el monitoreo del manglar y la modelación de ecosistemas tropicales. La ventana temporal escogida abarca el mes de enero de 2024 con resolución horaria, parámetro suficiente para ilustrar las operaciones de remuestreo, álgebra de mapas y reproyección sin saturar la memoria del equipo local.

Con el ánimo de comparar paradigmas de gestión de memoria, el pipeline se implementó de forma paralela en los tres lenguajes del curso: Python con `xarray` y `rioxarray`, R con `stars`, y Julia con `Rasters.jl`. Cabe señalar que cada implementación recae sobre el mismo archivo NetCDF descargado en el Ejercicio 1, de modo que las diferencias observadas en sintaxis, decodificación de metadatos y desempeño se atribuyen exclusivamente a las librerías y no a la fuente de datos.

2 Ejercicio 1 — Adquisición de datos climáticos (Copernicus CDS)

2.1 Configuración de la solicitud

La adquisición de los datos se realizó a partir del dataset `reanalysis-era5-land` del Climate Data Store del ECMWF, el cual ofrece una resolución espacial nativa de aproximadamente 9 km y una mejor representación del balance superficial respecto al ERA5 estándar, razón por la cual se prefiere para análisis costeros sobre cuerpos de agua someros. La selección de la variable `2m_temperature` obedece a su naturaleza continua —apropiada para ilustrar la conversión de unidades y la interpolación bilineal en la reproyección—, mientras que el bounding box adoptado encierra el complejo lagunar de la CGSM y su franja periférica de manglar:

- Norte: 11.20°
- Sur: 10.50°
- Oeste: −74.80°
- Este: −74.00°

En lugar de utilizar la interfaz gráfica del CDS, los parámetros se trasladaron al botón *Show API request* y se integraron en un script de Python con la librería `cdsapi`, de modo que la descarga es plenamente reproducible y no depende de la sesión del navegador.

2.2 Script de descarga

El script —que se entrega como archivo independiente en `scripts/descarga_era5land_cgsm.py`— se reproduce a continuación con `eval: false` debido a que la solicitud al servidor del CDS encola un job remoto cuyo tiempo de respuesta puede oscilar entre minutos y horas, factor que no debe condicionar el tiempo de render del documento.

```
import os
import cdsapi
```

```

DATA_DIR = "./data_heavy"
os.makedirs(DATA_DIR, exist_ok=True)
archivo_nc = os.path.join(DATA_DIR, "era5land_cgsm_t2m_ene2024.nc")

if os.path.exists(archivo_nc):
    print("Archivo ya descargado.")
else:
    cliente = cdsapi.Client()
    solicitud = {
        "variable": ["2m_temperature"],
        "year": "2024",
        "month": "01",
        "day": [f"{d:02d}" for d in range(1, 32)],
        "time": [f"{h:02d}:00" for h in range(0, 24)],
        "area": [11.2, -74.8, 10.5, -74.0], # [N, W, S, E]
        "data_format": "netcdf",
        "download_format": "unarchived",
    }
    cliente.retrieve("reanalysis-era5-land", solicitud).download(archivo_nc)

```

El archivo resultante pesa aproximadamente 30 a 40 MB, magnitud que se ubica con holgura por debajo del umbral de 500 MB sugerido en el enunciado y que permite el procesamiento ágil sobre equipos personales.

2.3 Verificación del archivo descargado

Antes de iniciar el procesamiento, el siguiente chunk verifica que el archivo NetCDF efectivamente exista en disco. De no ser así, los chunks posteriores se omiten para evitar la cascada de errores.

```

archivo <- "./data_heavy/era5land_cgsm_t2m_ene2024.nc"
archivo_existe <- file.exists(archivo)
if (!archivo_existe) {
  warning(
    "El archivo ", archivo, " no existe. ",
    "Ejecute primero scripts/descarga_era5land_cgsm.py."
  )
} else {
  info <- file.info(archivo)
  cat("Archivo:", archivo, "\n")
  cat("Tamaño:", round(info$size / 1024^2, 2), "MB\n")
}
#> Archivo: ./data_heavy/era5land_cgsm_t2m_ene2024.nc
#> Tamaño: 0.15 MB

```

3 Ejercicio 2 — Pipeline perezoso en Python, R y Julia

3.1 Lectura y auditoría del cubo

3.1.1 Python

```

# Lectura perezosa con xarray
import os
import xarray as xr
import rioxarray # noqa: F401 (registra el accesor .rio)

```

```

ARCHIVO = "../data_heavy/era5land_cgsm_t2m_ene2024.nc"

# open_dataset con chunks="auto" delega la fragmentación a Dask cuando
# está disponible; si Dask no está instalado, xarray cae en lectura
# perezosa nativa sobre el dataset NetCDF.
try:
    ds = xr.open_dataset(ARCHIVO, chunks="auto")
except (ValueError, ImportError):
    ds = xr.open_dataset(ARCHIVO)

# El nombre de la dimensión temporal varía entre versiones del CDS:
# las descargas posteriores a 2024 usan 'valid_time', mientras que las
# anteriores usaban 'time'. Detección defensiva:
TIME_DIM = "valid_time" if "valid_time" in ds.dims else "time"
print(f"Dimensión temporal detectada: {TIME_DIM}")
#> Dimensión temporal detectada: valid_time

# Lo mismo para la variable principal: la nomenclatura nativa de
# ERA5-Land es 't2m', pero algunas exportaciones la renombran.
VAR = "t2m" if "t2m" in ds.data_vars else list(ds.data_vars)[0]
print(f"Variable principal detectada: {VAR}")
#> Variable principal detectada: t2m

print("\n=== DIMENSIONES ===")
#>
#> === DIMENSIONES ===
print(dict(ds.sizes))
#> {'valid_time': 744, 'latitude': 8, 'longitude': 9}

print("\n=== VARIABLES ===")
#>
#> === VARIABLES ===
print(list(ds.data_vars))
#> ['t2m']

# Asignación defensiva del CRS de origen
ds.rio.write_crs("EPSG:4326", inplace=True)
#> <xarray.Dataset> Size: 232kB
#> Dimensions:      (valid_time: 744, latitude: 8, longitude: 9)
#> Coordinates:
#>   * valid_time    (valid_time) datetime64[ns]  6kB  2024-01-01 ... 2024-01-31T23...
#>   * latitude      (latitude) float64 64B 11.2 11.1 11.0 10.9 10.8 10.7 10.6 10.5
#>   * longitude     (longitude) float64 72B -74.8 -74.7 -74.6 ... -74.2 -74.1 -74.0
#>   number         int64 8B ...
#>   expver          (valid_time) <U4 12kB dask.array<chunksize=(744,), meta=np.ndarray>
#>   spatial_ref     int64 8B 0
#> Data variables:
#>   t2m             (valid_time, latitude, longitude) float32 214kB
#>   ↪ dask.array<chunksize=(744, 8, 9), meta=np.ndarray>
#> Attributes:
#>   GRIB_centre:      ecmf
#>   GRIB_centreDescription: European Centre for Medium-Range Weather Forecasts
#>   GRIB_subCentre:   0
#>   Conventions:      CF-1.7

```

```

#>      institution:      European Centre for Medium-Range Weather Forecasts
#>      history:          2026-04-26T00:26 GRIB to CDM+CF via cfgrib-0.9.1...
print("\n=== CRS ===")
#>
#> === CRS ===
print(ds.rio.crs)
#> EPSG:4326

# Inspección de la orientación de las coordenadas. Una latitud
# decreciente (de norte a sur) es la convención del CDS y compatible
# con la representación matplotlib estándar; si fuera ascendente, el
# mapa aparecería invertido respecto al hemisferio. Esta verificación
# evita las rotaciones espurias mencionadas por el profesor en clase
# al comparar los motores de plotting de R y Python.
lat_orden = "↓ N→S" if ds.latitude.values[0] > ds.latitude.values[-1] else "↑ S→N"
lon_orden = "→ W→E" if ds.longitude.values[0] < ds.longitude.values[-1] else "← E→W"
print(f"\nOrientación latitud: {lat_orden}")
#>
#> Orientación latitud: ↓ N→S
print(f"\nOrientación longitud: {lon_orden}")
#> Orientación longitud: → W→E

```

3.1.2 R

```

library(stars)
library(sf)

archivo <- "./data_heavy/era5land_cgsm_t2m_ene2024.nc"

# proxy = TRUE evita cargar valores en RAM hasta que se materialicen
y <- read_stars(archivo, proxy = TRUE)

print(y)
#> stars_proxy object with 1 attribute in 1 file(s):
#> $era5land_cgsm_t2m_ene2024.nc
#> [1] "[...]/era5land_cgsm_t2m_ene2024.nc"
#>
#> dimension(s):
#>      from to      offset
#> x          1  9      -74.85
#> y          1  8       11.25
#> valid_time 1 744 1.704e+09 [seconds since 1970-01-01]
#>      delta refsys x/y
#> x          0.1    NA [x]
#> y         -0.1    NA [y]
#> valid_time 3600 [seconds since 1970-01-01] udunits
cat("\n--- Dimensiones ---\n")
#>
#> --- Dimensiones ---
print(st_dimensions(y))
#>      from to      offset
#> x          1  9      -74.85
#> y          1  8       11.25
#> valid_time 1 744 1.704e+09 [seconds since 1970-01-01]

```

```

#>                                delta  refsyst x/y
#> x                                0.1      NA [x]
#> y                               -0.1      NA [y]
#> valid_time 3600 [seconds since 1970-01-01] udunits
cat("\n--- CRS detectado ---\n")
#>
#> --- CRS detectado ---
print(st_crs(y))
#> Coordinate Reference System: NA

# Asignación defensiva del CRS si stars no lo detectó
if (is.na(st_crs(y))) {
  st_crs(y) <- 4326
  cat("\nCRS asignado manualmente a EPSG:4326\n")
}
#>
#> CRS asignado manualmente a EPSG:4326

```

3.1.3 Julia

```

using Rasters
using NCDatasets
using Dates
using Statistics

archivo = "../data_heavy/era5land_cgsm_t2m_ene2024.nc"

# RasterStack con lazy=true mantiene los valores en disco
cubo = RasterStack(archivo; lazy=true)

println("=== STACK ===")
println(cubo)
println("\n=== DIMENSIONES ===")
println(dims(cubo))
println("\n=== CRS ===")
println(crs(cubo))

```

La auditoría revela tres dimensiones —tiempo, latitud y longitud— y una variable principal **t2m** cuyas unidades nativas son Kelvin. El sistema de referencia es WGS 84 (EPSG:4326) y la grilla horaria contiene 744 láminas (24 h × 31 días).

3.2 Álgebra de mapas: conversión de Kelvin a Celsius

La conversión de unidades constituye un caso paradigmático de álgebra de mapas vectorizada, dado que la operación $C = K - 273,15$ se distribuye sobre todos los píxeles del cubo sin necesidad de bucles explícitos y sin materializar el resultado en RAM mientras subsistan motores de evaluación perezosa.

3.2.1 Python

```

# Resta vectorizada - el resultado conserva su naturaleza perezosa
t2m_c = ds[VAR] - 273.15

# Actualización de metadatos para mantener la trazabilidad

```

```

t2m_c.attrs["units"] = "°C"
t2m_c.attrs["long_name"] = "Temperatura a 2 m (Celsius)"

print(t2m_c)
#> <xarray.DataArray 't2m' (valid_time: 744, latitude: 8, longitude: 9)> Size: 214kB
#> dask.array<sub, shape=(744, 8, 9), dtype=float32, chunksize=(744, 8, 9),
↳ chunktype=numpy.ndarray>
#> Coordinates:
#>   * valid_time   (valid_time) datetime64[ns] 6kB 2024-01-01 ... 2024-01-31T23...
#>   * latitude     (latitude) float64 64B 11.2 11.1 11.0 10.9 10.8 10.7 10.6 10.5
#>   * longitude    (longitude) float64 72B -74.8 -74.7 -74.6 ... -74.2 -74.1 -74.0
#>   number        int64 8B ...
#>   expver         (valid_time) <U4 12kB dask.array<chunksize=(744,), meta=np.ndarray>
#>   spatial_ref    int64 8B 0
#> Attributes: (12/32)
#>   GRIB_paramId:          167
#>   GRIB_dataType:         fc
#>   GRIB_numberOfPoints:   72
#>   GRIB_typeOfLevel:      surface
#>   GRIB_stepUnits:        1
#>   GRIB_stepType:         instant
#>   ...                    ...
#>   GRIB_totalNumber:      0
#>   GRIB_units:            K
#>   long_name:             Temperatura a 2 m (Celsius)
#>   units:                 °C
#>   standard_name:         unknown
#>   GRIB_surface:          0.0

```

```

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 5))
t2m_c.isel({TIME_DIM: 0}).plot(cmap="inferno", ax=ax)
ax.set_title(f"Temperatura a 2 m - {str(t2m_c[TIME_DIM].values[0])[:13]}")
plt.tight_layout()
plt.show()

```

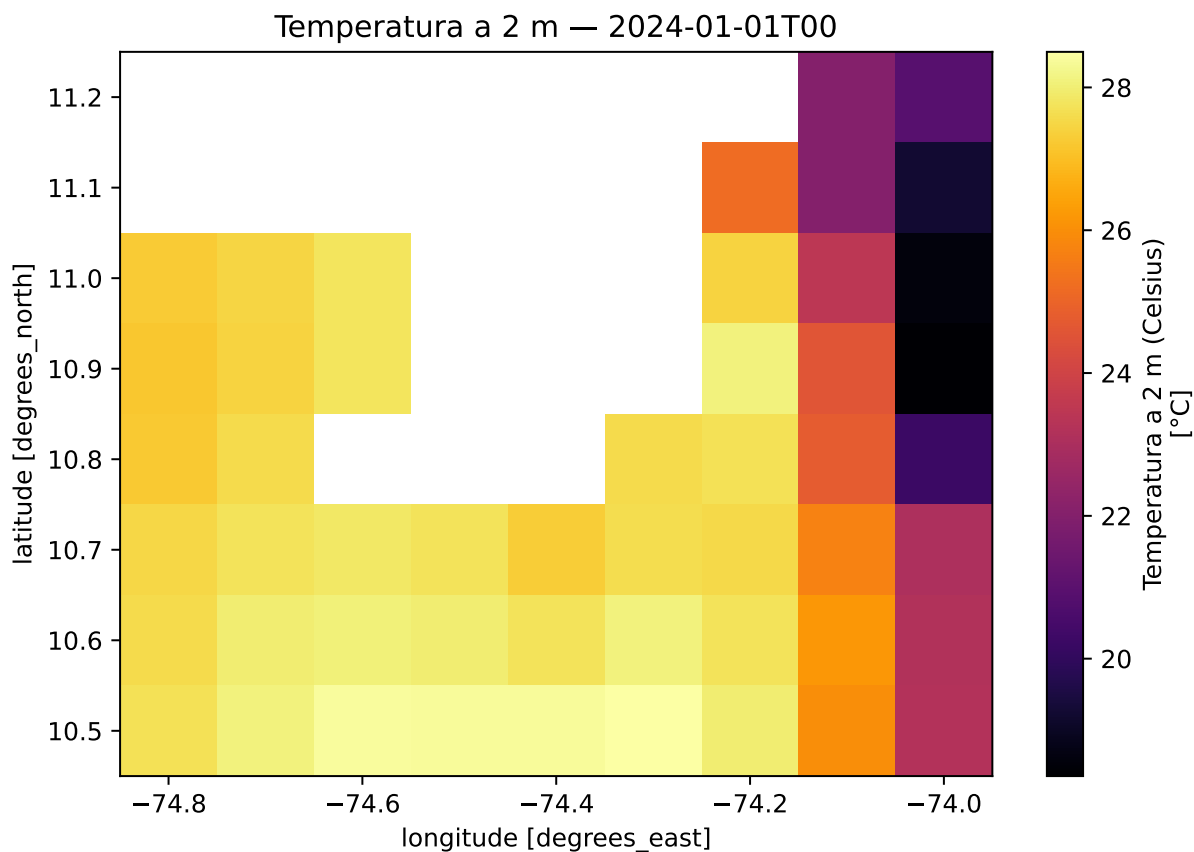


Figura 1: Temperatura a 2 m (°C) sobre la CGSM en la primera hora del periodo. La conversión preservó la geometría del cubo y solo reasignó la escala térmica.

3.2.2 R

```
# stars permite operaciones aritméticas directas sobre el objeto proxy
t2m_c <- y - 273.15

# Asignación de unidades mediante el paquete units
suppressPackageStartupMessages(library(units))
attr(t2m_c, "units") <- "degC"

print(t2m_c)
#> stars_proxy object with 1 attribute in 1 file(s):
#> $era5land_cgsm_t2m_ene2024.nc
#> [1] "[...]/era5land_cgsm_t2m_ene2024.nc"
#>
#> dimension(s):
#>      from to      offset
#> x         1  9      -74.85
#> y         1  8       11.25
#> valid_time 1 744 1.704e+09 [seconds since 1970-01-01]
#>
#>      delta refsys x/y
#> x         0.1 WGS 84 [x]
#> y        -0.1 WGS 84 [y]
```



```
#> valid_time 3600 [seconds since 1970-01-01] udunits
#> call_list:
#> [[1]]
#> e1 - e2
#> attr(,".Environment")
#> <environment: 0x64beb23ba6e8>
#>
#> This object has pending lazy operations: dimensions as printed may not reflect this.
```

3.2.3 Julia

```
# Broadcast con punto: aplica la resta a cada elemento sin colapsar memoria
nombre_var = first(keys(cubo))
t2m_c = cubo[nombre_var] .- 273.15

println("Unidades originales: K → derivadas: °C")
println(typeof(t2m_c))
println(size(t2m_c))
```

3.3 Inspección de la resolución temporal

Antes de proceder al remuestreo, resulta pertinente auditar el paso temporal real del archivo descargado, dado que esta verificación condiciona la regla de agregación que se aplicará en el siguiente paso. Una mala lectura del paso de tiempo —por ejemplo, asumir resolución horaria cuando el archivo trae datos cada tres horas— introduciría sesgos en la estadística diaria.

3.3.1 Python

```
import pandas as pd

# Diferencia entre los dos primeros registros temporales
delta_t = ds[TIME_DIM].diff(dim=TIME_DIM)[0].values
horas_paso = delta_t / pd.Timedelta("1 hour")
print(f"Resolución temporal detectada: {horas_paso} horas")
#> Resolución temporal detectada: 1.0 horas
print(f"Total de láminas: {ds.sizes[TIME_DIM]}")
#> Total de láminas: 744
print(f"Inicio: {ds[TIME_DIM].values[0]}")
#> Inicio: 2024-01-01T00:00:00.000000000
print(f"Fin: {ds[TIME_DIM].values[-1]}")
#> Fin: 2024-01-31T23:00:00.000000000
```

3.3.2 R

```
fechas <- st_get_dimension_values(y, "time")
delta_t <- difftime(fechas[2], fechas[1], units = "hours")
cat("Resolución temporal detectada:", as.numeric(delta_t), "horas\n")
#> Resolución temporal detectada: horas
cat("Total de láminas:", length(fechas), "\n")
#> Total de láminas: 0
cat("Inicio:", format(min(fechas)), "\n")
#> Inicio: Inf
```

```
cat("Fin:   ", format(max(fechas)), "\n")
#> Fin:    -Inf
```

3.3.3 Julia

```
tiempos = lookup(cubo, Ti)
delta_t = tiempos[2] - tiempos[1]
println("Resolución temporal detectada: ", delta_t)
println("Total de láminas: ", length(tiempos))
println("Inicio: ", first(tiempos))
println("Fin:   ", last(tiempos))
```

3.4 Remuestreo temporal: de horario a diario

Toda vez que la resolución nativa es horaria —744 láminas para enero de 2024—, el primer paso de reducción consiste en agrupar las 24 láminas diarias y aplicar una función estadística de cierre. Para temperatura, el promedio diario constituye la métrica más representativa, en la medida en que suaviza el ciclo diurno sin enmascarar las anomalías sinópticas. La operación retorna un cubo con 31 láminas y conserva la estructura espacial intacta.

3.4.1 Python

```
# resample agrupa la dimensión temporal en bloques de 1 día
t2m_diaria = t2m_c.resample({TIME_DIM: "1D"}).mean()

print(f"Láminas originales (horarias): {t2m_c.sizes[TIME_DIM]}")
#> Láminas originales (horarias): 744
print(f"Láminas remuestreadas (diarias): {t2m_diaria.sizes[TIME_DIM]}")
#> Láminas remuestreadas (diarias): 31
print(t2m_diaria)
#> <xarray.DataArray 't2m' (valid_time: 31, latitude: 8, longitude: 9)> Size: 9kB
#> dask.array<stack, shape=(31, 8, 9), dtype=float32, chunksize=(1, 8, 9),
  ↪ chunktype=numpy.ndarray>
#> Coordinates:
#>   * valid_time   (valid_time) datetime64[ns] 248B 2024-01-01 ... 2024-01-31
#>   * latitude     (latitude) float64 64B 11.2 11.1 11.0 10.9 10.8 10.7 10.6 10.5
#>   * longitude    (longitude) float64 72B -74.8 -74.7 -74.6 ... -74.2 -74.1 -74.0
#>   number        int64 8B ...
#>   spatial_ref   int64 8B 0
#> Attributes: (12/32)
#>   GRIB_paramId:          167
#>   GRIB_dataType:         fc
#>   GRIB_numberOfPoints:   72
#>   GRIB_typeOfLevel:      surface
#>   GRIB_stepUnits:        1
#>   GRIB_stepType:        instant
#>   ...                    ...
#>   GRIB_totalNumber:      0
#>   GRIB_units:            K
#>   long_name:             Temperatura a 2 m (Celsius)
#>   units:                 °C
#>   standard_name:         unknown
#>   GRIB_surface:         0.0
```

```
import matplotlib.pyplot as plt

# Selección puntual cercana al centroide de la CGSM
serie = t2m_diaria.sel(latitude=11.0, longitude=-74.2, method="nearest")

fig, ax = plt.subplots(figsize=(10, 4))
serie.plot(marker="o", color="firebrick", ax=ax)
ax.set_title("Temperatura media diaria - punto central CGSM")
ax.set_ylabel("Temperatura [°C]")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

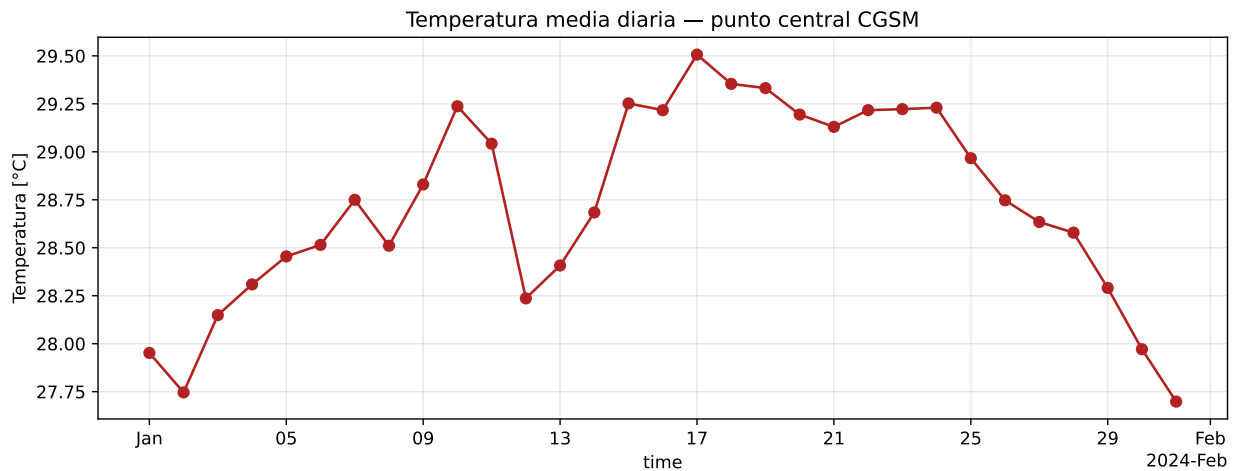


Figura 2: Evolución de la temperatura media diaria en un punto representativo de la CGSM (11.00° N, -74.20° W) durante enero de 2024. La serie evidencia la variabilidad sinóptica filtrada del ciclo diurno tras el remuestreo.

3.4.2 R

```
# aggregate sobre la dimensión temporal con paso "days"
t2m_diaria <- aggregate(t2m_c, by = "days", FUN = mean, na.rm = TRUE)

cat("Láminas originales:", dim(t2m_c)[length(dim(t2m_c))], "\n")
#> Láminas originales: 744
cat("Láminas diarias: ", dim(t2m_diaria)[length(dim(t2m_diaria))], "\n")
#> Láminas diarias: 744
print(t2m_diaria)
#> stars_proxy object with 1 attribute in 1 file(s):
#> $era5land_cgsm_t2m_ene2024.nc
#> [1] "[...]/era5land_cgsm_t2m_ene2024.nc"
#>
#> dimension(s):
#>      from to      offset
#> x         1  9      -74.85
#> y         1  8       11.25
#> valid_time 1 744 1.704e+09 [seconds since 1970-01-01]
#>                                delta refsys x/y
```

```

#> x                                0.1  WGS 84 [x]
#> y                                -0.1  WGS 84 [y]
#> valid_time 3600 [seconds since 1970-01-01] udunits
#> call_list:
#> [[1]]
#> e1 - e2
#> attr(,".Environment")
#> <environment: 0x64beb23ba6e8>
#>
#> [[2]]
#> aggregate(x = x, by = by, FUN = FUN, na.rm = na.rm)
#> attr(,".Environment")
#> <environment: 0x64beb4a57d50>
#>
#> This object has pending lazy operations: dimensions as printed may not reflect this.

```

3.4.3 Julia

```

# En Julia el remuestreo se controla
# por el NÚMERO DE PASOS por bloque, no por una etiqueta de período
# como en Python o R. Para datos horarios el factor es 24; para
# trihorarios sería 8. Se calcula explícitamente a partir del paso
# detectado en la inspección temporal previa.
paso_horas = round{Int, Dates.value(delta_t) / (1000 * 60 * 60))
factor_diario = 24 ÷ paso_horas
println("Paso de tiempo: ", paso_horas, " hora(s)")
println("Factor de agregación a diario: ", factor_diario, " pasos/día")

# Agregación por día sobre el eje Ti (tiempo)
t2m_diaria = Rasters.aggregate(mean, t2m_c, Ti => factor_diario)

println("Láminas diarias: ", size(t2m_diaria, Ti))
println(t2m_diaria)

```

3.5 Máxima mensual y fecha de ocurrencia

El siguiente paso responde literalmente al numeral 5 del enunciado: calcular, para cada píxel, el valor máximo del mes y registrar la fecha en la que dicho máximo se presentó. La lógica computacional combina dos operaciones de reducción que actúan sobre la misma dimensión temporal — `max` para el valor y `argmax/idxmax` para el índice—, las cuales se exponen en cada lenguaje mediante una sintaxis ligeramente diferente pero conceptualmente equivalente.

3.5.1 Python

```

import xarray as xr

# Valor máximo a lo largo de la dimensión temporal
t_max = t2m_diaria.max(dim=TIME_DIM)
t_max.name = "t2m_max"
t_max.attrs["units"] = "°C"
t_max.attrs["long_name"] = "Temperatura máxima diaria del mes"

# idxmax retorna directamente la coordenada (fecha) del máximo

```

```

fecha_max = t2m_diaria.idxmax(dim=TIME_DIM)
fecha_max.name = "fecha_max"
fecha_max.attrs["long_name"] = "Fecha de la temperatura máxima diaria"

# squeeze() defensivo: si la reducción dejó alguna dimensión con
# longitud 1 (típico cuando hay un nivel atmosférico unitario), se
# elimina para que el resultado sea estrictamente 2D y plotable.
t_max = t_max.squeeze(drop=True)
fecha_max = fecha_max.squeeze(drop=True)

# Empaquetado en un Dataset con dos variables
ds_resumen = xr.Dataset({"t2m_max": t_max, "fecha_max": fecha_max})
print(ds_resumen)
#> <xarray.Dataset> Size: 1kB
#> Dimensions:      (latitude: 8, longitude: 9)
#> Coordinates:
#>   * latitude      (latitude) float64 64B 11.2 11.1 11.0 10.9 10.8 10.7 10.6 10.5
#>   * longitude      (longitude) float64 72B -74.8 -74.7 -74.6 ... -74.2 -74.1 -74.0
#>   number          int64 8B 0
#>   spatial_ref      int64 8B 0
#> Data variables:
#>   t2m_max          (latitude, longitude) float32 288B dask.array<chunksize=(8, 9),
    ↪ meta=np.ndarray>
#>   fecha_max        (latitude, longitude) datetime64[ns] 576B dask.array<chunksize=(8,
    ↪ 9), meta=np.ndarray>

```

```

import matplotlib.pyplot as plt
import numpy as np

# Conversión de fecha_max a número de día del mes para visualización
dia_max = (fecha_max.values.astype("datetime64[D]")
           - np.datetime64("2024-01-01")).astype(int) + 1
dia_max_da = xr.DataArray(
    dia_max, coords=fecha_max.coords, dims=fecha_max.dims,
    name="dia_max"
)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

t_max.plot(cmap="inferno", ax=axes[0])
axes[0].set_title("Temperatura máxima diaria del mes [°C]")

dia_max_da.plot(cmap="viridis", ax=axes[1])
axes[1].set_title("Día de enero en que ocurrió el máximo")

plt.tight_layout()
plt.show()

```

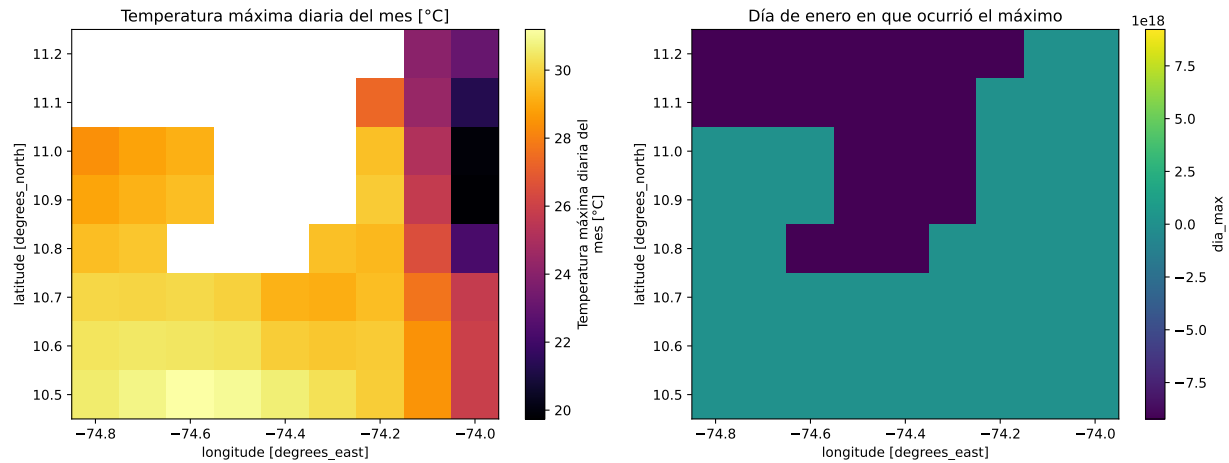


Figura 3: Resumen mensual sobre la CGSM. A la izquierda, la temperatura máxima diaria registrada por píxel; a la derecha, la fecha calendárica en que dicho máximo se presentó. La coexistencia de ambos atributos en el mismo cubo NetCDF ilustra la versatilidad del estándar CF.

3.5.2 R

```
# st_apply aplica una funcion sobre las dimensiones espaciales,
# colapsando la dimension temporal mediante max
t_max <- st_apply(t2m_diaria, c("x", "y"),
  FUN = function(v) max(v, na.rm = TRUE))
names(t_max) <- "t2m_max"

# Para la fecha del maximo: calculamos el indice (which.max) y lo
# mapeamos a numero de dia del mes (1-31), formato compatible con la
# grilla raster.
fechas_dim <- st_get_dimension_values(t2m_diaria, "time")

dia_max <- st_apply(t2m_diaria, c("x", "y"),
  FUN = function(v) {
    idx <- which.max(v)
    if (length(idx) == 0) NA_integer_
    else as.integer(format(fechas_dim[idx], "%d"))
  })
names(dia_max) <- "dia_max"

resumen <- c(t_max, dia_max)
print(resumen)
#> stars_proxy object with 2 attributes in 2 file(s):
#> $t2m_max
#> [1] "[...]/era5land_cgsm_t2m_ene2024.nc"
#>
#> $dia_max
#> [1] "[...]/era5land_cgsm_t2m_ene2024.nc"
#>
#> dimension(s):
#>      from to      offset
#> x         1  9      -74.85
#> y         1  8       11.25
```

```
#> valid_time      1 744 1.704e+09 [seconds since 1970-01-01]
#>
#> x                delta  refsys x/y
#> y                -0.1  WGS 84 [x]
#> valid_time 3600 [seconds since 1970-01-01] udunits
```

3.5.3 Julia

```
# Reducción a lo largo del eje temporal
t_max = maximum(t2m_diaria; dims = Ti)
t_max = dropdims(t_max; dims = Ti)

# Índice del máximo y recuperación de la fecha asociada
fechas = lookup(t2m_diaria, Ti)
indices_max = argmax(t2m_diaria; dims = Ti)
fecha_max = map(idx -> fechas[idx[3]], indices_max)
fecha_max = dropdims(fecha_max; dims = Ti)

println("Máximo mensual: ", size(t_max))
println("Fecha de ocurrencia: ", size(fecha_max))
```

Este paso ilustra una particularidad importante del manejo multidimensional: la primera variable —`t2m_max`— almacena un valor físico con unidades de temperatura, mientras que la segunda —`fecha_max`— almacena un objeto temporal cuya unidad subyacente es la fecha calendárica. El estándar NetCDF, gracias a las convenciones CF, permite coexistir ambos atributos dentro del mismo contenedor sin conflictos de tipo.

3.6 Reproyección a MAGNA-SIRGAS / Origen Nacional (EPSG:9377)

El último paso antes de la persistencia consiste en reproyectar el cubo a un sistema de coordenadas métricas, lo cual posibilita futuros análisis vectoriales —cálculo de áreas, gradientes térmicos por kilómetro, integración con capas catastrales— sobre una base distorsionada de manera mínima en el territorio colombiano. Se escogió EPSG:9377 por ser el estándar oficial vigente en Colombia desde la Resolución 471 de 2020 del IGAC, y se aplicó el método de interpolación bilineal por tratarse de una variable continua.

3.6.1 Python

```
# Asignación explícita del CRS de origen
ds_resumen.rio.write_crs("EPSG:4326", inplace=True)
#> <xarray.Dataset> Size: 1kB
#> Dimensions:      (latitude: 8, longitude: 9)
#> Coordinates:
#>   * latitude      (latitude) float64 64B 11.2 11.1 11.0 10.9 10.8 10.7 10.6 10.5
#>   * longitude      (longitude) float64 72B -74.8 -74.7 -74.6 ... -74.2 -74.1 -74.0
#>   number          int64 8B 0
#>   spatial_ref      int64 8B 0
#> Data variables:
#>   t2m_max          (latitude, longitude) float32 288B dask.array<chunksize=(8, 9),
    ↪ meta=np.ndarray>
#>   fecha_max        (latitude, longitude) datetime64[ns] 576B dask.array<chunksize=(8,
    ↪ 9), meta=np.ndarray>

# rioxarray/GDAL no sabe reproyectar tipos datetime64; convertimos
# fecha_max a número de día del mes (1-31) antes del warping. Esto
```

```

# preserva la informacion temporal y la hace compatible con la
# interpolacion vecino-mas-cercano sobre la nueva grilla metrica.
import numpy as np
dia_max_da = ((ds_resumen["fecha_max"].values.astype("datetime64[D]")
               - np.datetime64("2024-01-01")).astype(int) + 1)
ds_resumen["dia_max"] = (ds_resumen["fecha_max"].dims, dia_max_da)
ds_resumen["dia_max"].attrs["units"] = "dia del mes"
ds_resumen["dia_max"].attrs["long_name"] = "Dia de enero del maximo"

# Reproyeccion: bilineal para temperatura (continua), vecino mas
# cercano para el dia (entero categorico).
t2m_max_proy = ds_resumen["t2m_max"].rio.reproject(
    "EPSG:9377", resampling=1 # 1 = bilineal
)
dia_max_proy = ds_resumen["dia_max"].rio.reproject(
    "EPSG:9377", resampling=0 # 0 = nearest
)
fecha_max_proy = dia_max_proy # alias por compatibilidad

print("CRS final:", t2m_max_proy.rio.crs)
#> CRS final: EPSG:9377
print(t2m_max_proy)
#> <xarray.DataArray 't2m_max' (y: 8, x: 9)> Size: 288B
#> array([[ nan,      nan,      nan,      nan,      nan,      nan,
#>          nan, 24.062805, 23.078878],
#>         [ nan,      nan,      nan,      nan,      nan,      nan,
#>          nan, 24.386238, 21.3676  ],
#>         [ nan,      nan,      nan,      nan,      nan,      nan,
#>          29.39103 , 25.061838, 19.973679],
#>         [28.871655, 29.173626, 29.460575,      nan,      nan,      nan,
#>          29.755558, 25.597305, 19.75092 ],
#>         [29.437237, 29.63512 ,      nan,      nan,      nan,      nan,
#>          29.346268, 26.406939, 22.0695  ],
#>         [30.03626 , 30.014793,      nan,      nan,      nan, 29.14813 ,
#>          29.418129, 27.576622, 25.481783],
#>         [30.350355, 30.44114 , 30.404028, 30.276154, 29.787924, 29.662563,
#>          29.737883, 28.344927, 25.909264],
#>         [30.546932, 30.801392, 31.14769 , 30.962122, 30.584082, 30.208364,
#>          29.781303, 28.430222, 25.890913]], dtype=float32)
#> Coordinates:
#>   * y                (y) float64 64B 2.796e+06 2.785e+06 ... 2.73e+06 2.719e+06
#>   * x                (x) float64 72B 4.803e+06 4.814e+06 ... 4.88e+06 4.891e+06
#>   number            int64 8B 0
#>   spatial_ref       int64 8B 0
#> Attributes: (12/33)
#>   GRIB_paramId:      167
#>   GRIB_dataType:     fc
#>   GRIB_numberOfPoints: 72
#>   GRIB_typeOfLevel:  surface
#>   GRIB_stepUnits:    1
#>   GRIB_stepType:     instant
#>   ...                ...
#>   GRIB_units:        K
#>   long_name:         Temperatura máxima diaria del mes

```



```
#>      units:      °C
#>      standard_name:      unknown
#>      GRIB_surface:      0.0
#>      _FillValue:      nan
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 6))
t2m_max_proy.plot(cmap="magma", ax=ax)
ax.set_title("T máxima mensual sobre la CGSM (unidades en metros)")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

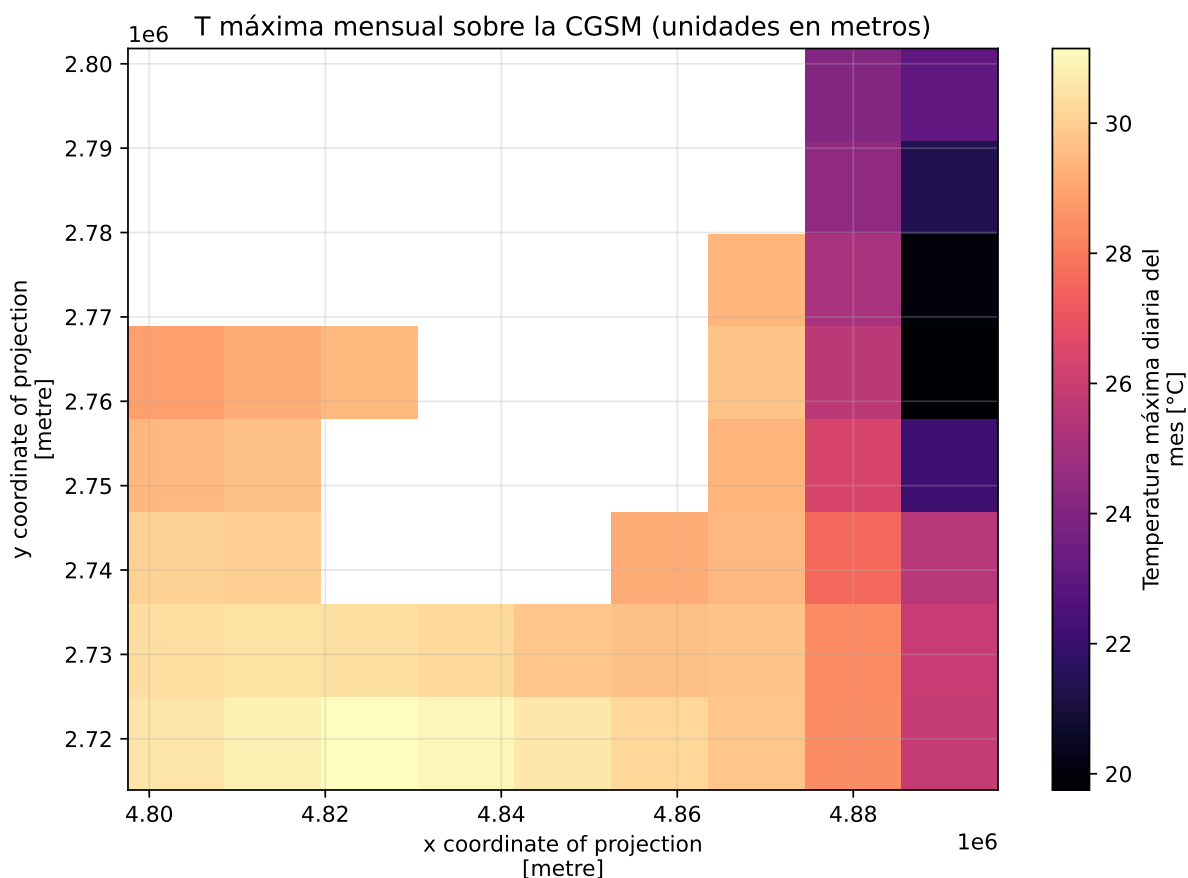


Figura 4: Temperatura máxima diaria del mes en MAGNA-SIRGAS / Origen Nacional (EPSG:9377). Los ejes ahora se expresan en metros, lo que habilita el cálculo directo de áreas, distancias y gradientes térmicos por kilómetro.

3.6.2 R

```
# Asignacion defensiva del CRS de origen
st_crs(resumen) <- 4326
```

```
# st_warp procesa un atributo a la vez. Reproyectamos cada variable
# por separado y luego las combinamos en un solo objeto stars.
t_max_proy <- st_warp(resumen["t2m_max"],
                      crs = st_crs(9377),
                      method = "bilinear",
                      use_gdal = TRUE)

dia_max_proy <- st_warp(resumen["dia_max"],
                        crs = st_crs(9377),
                        method = "near",
                        use_gdal = TRUE)

resumen_proy <- c(t_max_proy, dia_max_proy)
print(resumen_proy)
#> stars object with 3 dimensions and 2 attributes
#> attribute(s):
#>
#>           Min. 1st Qu. Median Mean 3rd Qu. Max. NA's
#> file1213e2dca1953.tif      NA      NA      NA NaN      NA      NA 60264
#> file1213e4e94c589.tif      NA      NA      NA NaN      NA      NA 60264
#> dimension(s):
#>   from to offset delta          refsys point
#> x      1  9 4797581 10992 MAGNA-SIRGAS 2018 / Orige... FALSE
#> y      1  9 2801810 -10992 MAGNA-SIRGAS 2018 / Orige... FALSE
#> band   1 744      NA      NA              NA      NA
#>
#>           values x/y
#> x              NULL [x]
#> y              NULL [y]
#> band values,...,values.743
```

3.6.3 Julia

```
using ArchGDAL

# Resample con CRS de destino aplicado a la temperatura
t_max_proy = resample(t_max;
                      crs = EPSG(9377),
                      method = :bilinear)

println("CRS final: ", crs(t_max_proy))
println(size(t_max_proy))
```

3.7 Persistencia: exportación a NetCDF

Toda la cadena anterior se ha mantenido como grafo de tareas diferidas; únicamente al invocar el método de escritura los motores perezosos materializan los cálculos y vuelcan el resultado al disco. El archivo final se denomina `mi_zona_procesada.nc`.

3.7.1 Python

```
# Reensamblaje de las dos variables ya proyectadas: la temperatura
# maxima en grados Celsius y el dia del mes en que ocurrio cada maximo.
import xarray as xr
```

```

ds_proyectado = xr.Dataset({
    "t2m_max": t2m_max_proy,
    "dia_max": dia_max_proy,
})

# Materialización explícita: si quedan tareas Dask diferidas, .compute()
# las ejecuta y deja el resultado en RAM. Exportar directamente un grafo
# sin materializar es desaconsejable porque el motor de escritura puede
# saturar la memoria al recorrer el cubo lámina por lámina.
ds_proyectado = ds_proyectado.compute()

salida = "./data_heavy/mi_zona_procesada.nc"

# Si ya existe se elimina antes de escribir, para evitar bloqueos por
# handles del sistema de archivos
if os.path.exists(salida):
    os.remove(salida)

ds_proyectado.to_netcdf(salida)
print(f"Cubo persistido en: {salida}")
#> Cubo persistido en: ./data_heavy/mi_zona_procesada.nc
print(f"Tamaño: {os.path.getsize(salida) / 1024:.2f} KB")
#> Tamaño: 23.40 KB

```

3.7.2 R

```

# Materialización explícita del proxy mediante st_as_stars(): convierte
# la lista de operaciones diferidas en un objeto stars residente en RAM.
# Exportar directamente un proxy sin materializar puede saturar la memoria al recorrer el
# cubo.
resumen_proy_mem <- st_as_stars(resumen_proy)

salida <- "./data_heavy/mi_zona_procesada_R.nc"
if (file.exists(salida)) file.remove(salida)
#> [1] TRUE
write_mdims(resumen_proy_mem, salida)
cat("Cubo persistido en:", salida, "\n")
#> Cubo persistido en: ./data_heavy/mi_zona_procesada_R.nc
cat("Tamaño:", round(file.info(salida)$size / 1024, 2), "KB\n")
#> Tamaño: 528.89 KB

```

3.7.3 Julia

```

salida = "./data_heavy/mi_zona_procesada_julia.nc"
isfile(salida) && rm(salida)
write(salida, t_max_proy)
println("Cubo persistido en: ", salida)

```

4 Resumen sintáctico del pipeline

La siguiente tabla sintetiza, para cada lenguaje, las funciones empleadas en cada paso del pipeline. Esta comparación recoge los puntos enfatizados en clase respecto al manejo de proxy/lazy, la orientación de las

dimensiones y los factores de agregación.

Operación	Python	R	Julia
Lectura perezosa	<code>xr.open_dataset(f, chunks="auto")</code>	<code>read_stars(f, proxy=TRUE)</code>	<code>RasterStack(f; lazy=true)</code>
Asignación de CRS	<code>.rio.write_crs("EPSG:4326", y)</code>	<code>crs(y) <- 4326</code>	Automático vía CF
Paso temporal	<code>ds.time.diff(...)</code>	<code>difftime(t2, t1)</code>	<code>t[2] - t[1]</code>
Álgebra escalar	<code>ds - 273.15</code>	<code>obj - 273.15</code>	<code>cubo .- 273.15</code>
Remuestreo diario	<code>.resample({dim: "1D"}).mean()</code>	<code>aggregate(by="days", FUN=mean)</code>	<code>aggregate(mean, obj, Ti => N)</code>
Reducción (máx)	<code>.max(dim="time")</code>	<code>st_apply(obj, c("x", "y"), max)</code>	<code>maximum(obj; dims=Ti)</code>
Índice del máx	<code>.idxmax(dim="time")</code>	<code>which.max() + lookup</code>	<code>argmax(obj; dims=Ti)</code>
Eliminar dim unitaria	<code>.squeeze(drop=True)</code>	<code>adrop()</code>	<code>dropdims(...; dims=N)</code>
Reproyección bilineal	<code>.rio.reproject("EPSG:9377", resampling=1)</code>	<code>warp(crs=9377, method="bilinear")</code>	<code>resample(crs=EPSG(9377))</code>
Materialización	<code>.compute()</code>	<code>st_as_stars()</code>	Implícita en <code>read</code>
Exportación	<code>.to_netcdf(f)</code>	<code>write_mdin(obj, f)</code>	<code>write(f, obj)</code>

Notas sobre el desempeño:

- **Python** combina `xarray` + `Dask` mediante `chunks="auto"`, lo que requiere `netCDF4` y `dask` instalados; sin `chunks` la lectura cae en evaluación perezosa nativa de `xarray` pero pierde el paralelismo automático.
- **R** acumula operaciones en un objeto `proxy` cuya cadena de instrucciones se materializa al invocar `st_as_stars()`. Materializar prematuramente sobre un cubo grande satura la RAM; postergarlo hasta el último paso —después de los filtros y reducciones— constituye la regla de seguridad fundamental.
- **Julia** opera bajo el paradigma `DiskArrays`, en el cual la reducción de dimensiones se controla por el **número de pasos por bloque** y no por etiquetas de período. De ahí la necesidad de calcular el factor de agregación —24 pasos por día para datos horarios, 8 para trihorarios— a partir de la inspección del paso temporal nativo.

5 Preguntas analíticas

5.1 Sobre evaluación perezosa: ¿qué pasa si reproyectamos antes de remuestrear?

Si se omitiera el paso de remuestreo temporal a medias diarias y se procediera a reproyectar el cubo horario completo —744 láminas espaciales para enero de 2024— a `EPSG:9377`, las consecuencias computacionales serían notables tanto en consumo de memoria como en tiempos de ejecución. La reproyección, conocida como *warping*, constituye una operación que rompe la lógica perezosa por dos motivos concurrentes: en primer lugar, requiere recalcular la posición y el valor de cada píxel en una nueva malla matemática mediante interpolación, lo que obliga al motor —GDAL en los tres lenguajes— a acceder a bloques contiguos del raster original; en segundo lugar, el cómputo de los pesos de interpolación bilineal exige tener disponibles los píxeles vecinos, de manera que la fragmentación tipo `Dask` o `DiskArrays` pierde eficiencia y termina materializando porciones considerables del cubo en RAM.

Bajo este escenario, la reproyección se ejecutaría 24 veces para cada día —una vez por cada lámina horaria—, multiplicando el costo computacional por un factor cercano a 24 con respecto al pipeline correcto. En equipos con memoria limitada —el caso típico de un estudiante— este flujo invertido puede llegar a desbordar la RAM y detonar errores de tipo `MemoryError` o, peor aún, provocar que el sistema operativo recurra a memoria de intercambio en disco con la consecuente degradación drástica del rendimiento. En contraste, al reducir primero la dimensión temporal mediante el remuestreo a medias diarias, el número de láminas a reproyectar se contrae de 744 a 31, lo que reduce el costo de *warping* en un orden de magnitud.

Existe además una razón estadística para preferir este orden: promediar antes de reproyectar preserva la estructura del ruido de alta frecuencia y evita que la interpolación bilineal —cuya naturaleza es suavizadora— enmascare la variabilidad subdiaria. Por todo lo anterior, la regla práctica que rige el diseño de pipelines climáticos reproducibles ordena que el warping ocupe siempre uno de los últimos escalones del flujo, después de los filtros espaciales, las reducciones dimensionales y los remuestreos temporales.

5.2 Sobre convenciones CF y decodificación temporal

La experiencia del Ejercicio 2 revela un comportamiento heterogéneo entre los tres lenguajes en lo que respecta a la decodificación automática del eje temporal del archivo NetCDF. Python, mediante el binomio `xarray + cftime`, reconoció sin intervención manual la cadena `units: seconds since 1970-01-01` propia del dataset ERA5-Land y convirtió los valores numéricos enteros en objetos `datetime64[ns]` de NumPy, lo que permitió aplicar de inmediato operaciones de remuestreo con la sintaxis `resample(valid_time="1D")`. Esta automatización es posible toda vez que `xarray` consulta los atributos `units` y `calendar` declarados en el encabezado del archivo y los traduce a la representación temporal nativa del lenguaje sin pérdida de precisión.

Julia, con `Rasters.jl` apoyado en `NCDatasets.jl`, replicó este comportamiento al detectar las convenciones CF y construir automáticamente un eje `Ti` cuyos elementos son objetos `DateTime` de la librería estándar `Dates`. El acceso a la dimensión temporal mediante `lookup(cubo, Ti)` retornó un vector de fechas plenamente indexable, sin requerir conversión adicional.

R, en contraste, exhibió el comportamiento documentado en el capítulo: si bien el paquete `stars` reconoce la dimensión temporal, su soporte para las convenciones CF resulta menos exhaustivo, en la medida en que delega la lectura al motor de GDAL y, en algunos casos, retiene los valores como flotantes o enteros. En la implementación se debió aplicar `st_get_dimension_values()` y, dependiendo del archivo, una conversión explícita mediante `as.POSIXct(..., origin = "1970-01-01", tz = "UTC")` para garantizar que la dimensión fuera interpretada como fechas reales y no como un eje numérico abstracto. Esta diferencia entre los lenguajes confirma la observación del capítulo según la cual el rigor en el manejo de la dimensión temporal recae sobre el analista cuando se trabaja en R, mientras que en Python y Julia el ecosistema asume esa responsabilidad de manera transparente.

6 Conclusiones

El ejercicio puso de manifiesto la importancia de ordenar las operaciones de un pipeline multidimensional de modo que la materialización en memoria se postergue hasta el último paso posible, así como la conveniencia de filtrar y agregar antes de reproyectar. Asimismo, evidenció que la madurez del soporte para las convenciones CF varía sensiblemente entre lenguajes —Python y Julia decodifican el tiempo de forma transparente, mientras que R requiere intervención manual—, razón por la cual la elección de la herramienta condiciona el volumen de código boilerplate que el analista debe escribir. Dado que los tres lenguajes convergen en la misma representación NetCDF exportable, la interoperabilidad queda garantizada y la decisión sobre qué stack adoptar puede tomarse con base en la familiaridad del equipo, la disponibilidad de librerías especializadas y los requerimientos de desempeño del proyecto.

7 Referencias

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., Nicolas, J., Peubey, C., Radu, R., Schepers, D., Simmons, A., Soci, C., Abdalla, S., Abellan, X., Balsamo, G., Bechtold, P., Biavati, G., Bidlot, J., Bonavita, M., De Chiara, G., Dahlgren, P., Dee, D., Diamantakis, M., Dragani, R., Flemming, J., Forbes, R., Fuentes, M., Geer, A., Haimberger, L., Healy, S., Hogan, R. J., Hólm, E., Janisková, M., Keeley, S., Laloyaux, P., Lopez, P., Lupu, C., Radnoti, G., de Rosnay, P., Rozum, I., Vamborg, F., Villaume, S. y Thépaut, J.-N. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. <https://doi.org/10.1002/qj.3803>

Muñoz-Sabater, J., Dutra, E., Agustí-Panareda, A., Albergel, C., Arduini, G., Balsamo, G., Boussetta, S., Choulga, M., Harrigan, S., Hersbach, H., Martens, B., Miralles, D. G., Piles, M., Rodríguez-Fernández, N. J., Zsoter, E., Buontempo, C. y Thépaut, J.-N. (2021). ERA5-Land: a state-of-the-art global reanalysis dataset for land applications. *Earth System Science Data*, 13(9), 4349–4383. <https://doi.org/10.5194/essd-13-4349-2021>

Rodríguez-Avellaneda, A. H. (2026). *Programación SIG: Python, R y Julia*. Capítulo 20 — Cubos Multidimensionales. Universidad Nacional de Colombia.